Digital Systems

Lecture 8

Introducing I/O

Mark van der Wilk

Hilary Term 2026

University of Oxford
Department of Computer Science

Following the course of past years by Mike Spivey

Question from Last Lecture

Other load/store instructions…

`ldr` and `str` deal in 32-bit values, the size of a register. But there are also

- `ldrb` and `strb` for 8-bit values (useful for strings).
- `ldrh` and `strh` for 16-bit values.
- `ldrsb` and `ldrsh` to load 8- or 16-bit values with sign extension. Why is this necessary?

Aside: Buffer Overrun Attacks

- See notes. Not examinable this year (sadly!).
- Shows how, in many programs that communicate with the outside world, to make it execute arbitrary code.
- Key principle behind lots of exploits that allow hacking into computer systems.
- Highly recommend looking at it, at some point (… perhaps after term or in summer)

Digital Systems

Lecture 8

Introducing I/O

Mark van der Wilk

Hilary Term 2026

University of Oxford
Department of Computer Science

Following the course of past years by Mike Spivey

Today

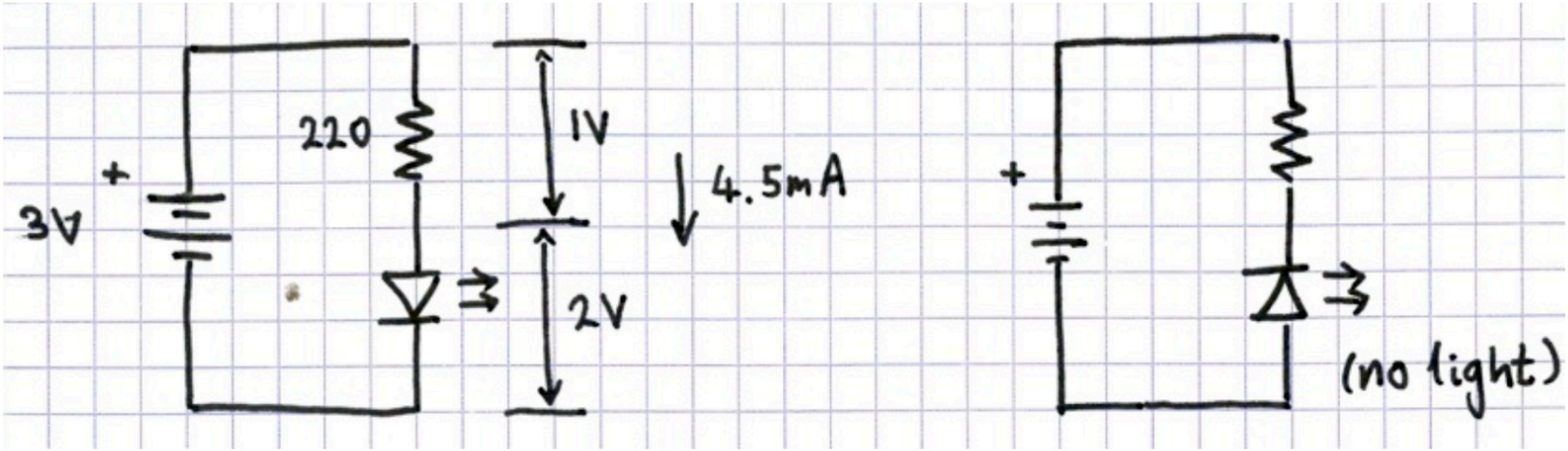
? How can we control LEDs on the BBC micro:bit?

But before we can answer this, we need to know a bit about how LEDs work, *electronically*.

Electronics of LEDs

Discuss on board: I-V char of LEDs

- Design: $R = 222\Omega, I \approx 4.5\text{mA}$.
- General Purpose In/Out (GPIO) pins on chip act as voltage source.

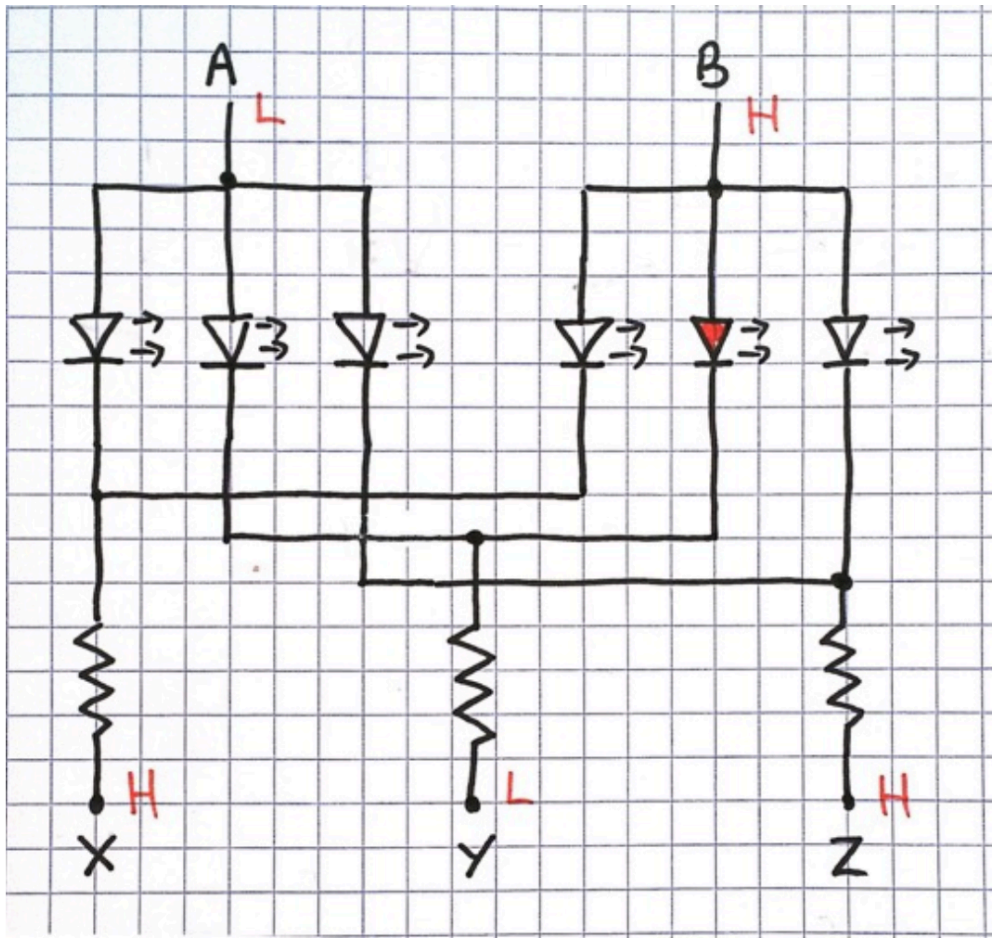


⚠ Limited GPIO pins on chip

We could connect one LED per GPIO pin…
but nRF51822 only has 31 GPIO pins!

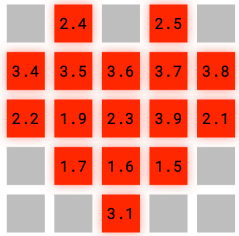
LED Multiplexing

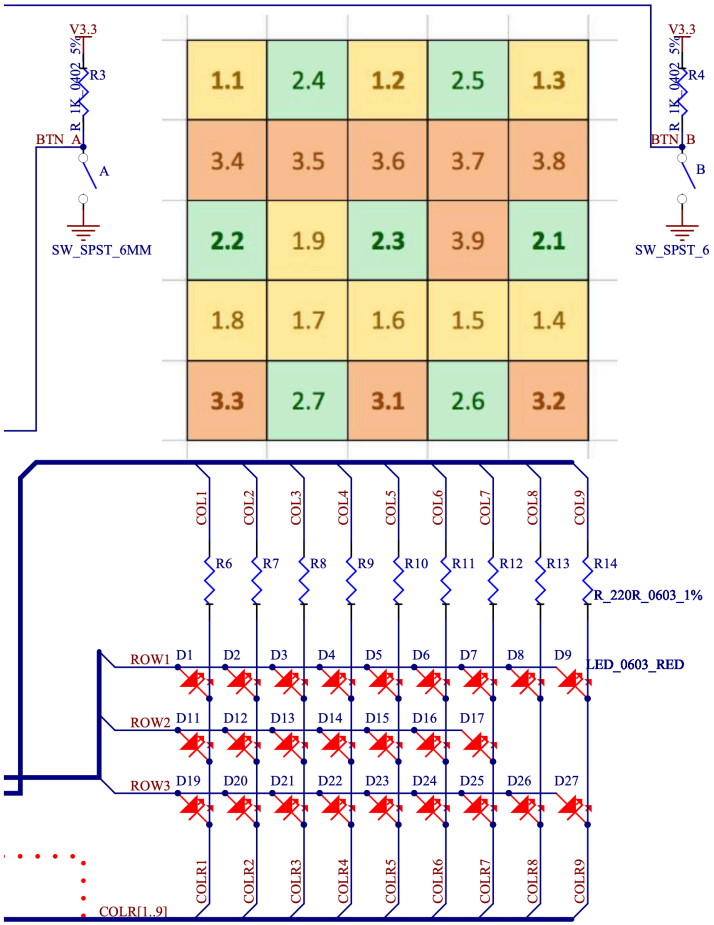
- A, B connected to anode (+ side of LED)
- X, Y, Z connected to cathode (- side of LED)
- Setting cathode-controlling GPIO to high, switches off all LEDs connected to it.
- LEDs no longer *independently* controllable.
- Can still make arbitrary patterns by switching on/off quickly.



LED Multiplexing on the BBC

- BBC micro:bit has 25 multiplexed LEDs, in 5 physical rows and columns.
- *Electronically* arranged in 3 groups of 9.
- Can display patters by controlling each group in sequence:
  - Row 1: 5, 6, 7, 9
  - Row 2: 1, 2, 3, 4, 5
  - Row 3: 1, 4, 5, 6, 7, 8, 9





Questions

Consider

A GPIO ports connected to LED anodes, and

C GPIO ports connected to LED cathodes.

How many LEDs can you control in total?

How many LEDs can be *independently* controlled?

Controlling GPIO outputs

- GPIO controls voltage.
- Peripherals are *memory mapped*,
  i.e. controlled by writing to a memory location.

```
GPIO_DIR 0x50000514          Controls in/out direction
GPIO_OUT 0x50000504          High or low for each pin
```

For example:

```
ldr r0, =0x50000504
ldr r1, =0x5fb0
str r1, [r0]
```

or in C:

```
#include "hardware.h"
...
GPIO_OUT = 0x5fb0
```

Lighting a Single LED

LEDs are connected as:

```
     ...|R3 R2 R1 C9|C8 C7 C6 C5|C4 C3 C2 C1|0000
bits     15----13 12-----------------------4
```

So lighting LED 5 in row 2..:

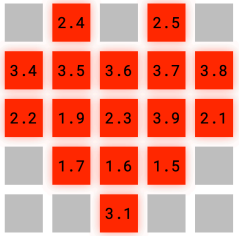
```
...   |R3 R2 R1 C9|C8 C7 C6 C5|C4 C3 C2 C1|0000
...00 | 0  1  0  1| 1  1  1  1| 1  0  1  1|0000
       5           f           b           0
```

```
GPIO_OUT = 0x5fb0
```

GPIO Bit Patterns

LEDs are connected as:

```
     ...|R3 R2 R1 C9|C8 C7 C6 C5|C4 C3 C2 C1|0000
bits     15----13 12-----------------------4
```



- Row 1: 5, 6, 7, 9
- Row 2: 1, 2, 3, 4, 5
- Row 3: 1, 4, 5, 6, 7, 8, 9

```
...    |R3 R2 R1 C9|C8 C7 C6 C5|C4 C3 C2 C1|0000
...00 | 1  0  0  0| 0  0  0  0| 0  1  1  0|0000
        8           0           6           0
```

Multiplexing GPIO

```
while (1) {
    GPIO_OUT = 0x28f0;
    delay(JIFFY);
    GPIO_OUT = 0x5e00;
    delay(JIFFY);
    GPIO_OUT = 0x8060;
    delay(JIFFY);
}
```

Use say JIFFY = 5000 for 67 updates/sec.

Frame Function

```
static const unsigned heart[] = {  0x28f0, 0x5e00, 0x8060  }
/* frame -- show three rows n times */
void frame(const unsigned *img, int n) {
  while (n > 0) {
    for (int p = 0; p < 3; p++) {
      GPIO_OUT = img[p];
      delay(JIFFY);
    }
    n--;
  }
}
```

Implementing Delay

```
void delay(unsigned usec) {
  unsigned n = 2 * usec;
  while (n > 0) {
    nop(); nop(); nop();
    n--;
  }
}
```

- Experiment shows that each iteration takes 8 cycles = 0.5µs at 16MHz.
- Different numbers needed on V2 at 64MHz.
- Different numbers needed if compiler changes.
- … or if anything changes!

How is `GPIO_OUT` implemented?

We want to make C write to a specific memory location! Pointers.

```
(* (volatile unsigned *) 0x50000504) = ...
```

Use C preprocessor to do this.

```
#define _REG(ty, addr)          (* (ty volatile *) addr)
#define GPIO_OUT                _REG(unsigned, 0x50000504)
```

Pushbuttons

Board: Circuit for pushbutton (pull-up resistor)

Must configure a pin to be an input:

```
SET_FIELD(GPIO_PINCNF[BUTTON_A], GPIO_PINCNF_INPUT,
GPIO_INPUT_Connect);
```

See `hardware.h`.

```
unsigned x = GPIO_IN;
if (GET_BIT(x, BUTTON_A) == 0 || GET_BIT(x, BUTTON_B) == 0) {
  // A button is pressed
}
```

Summary

- How LEDs work electrically.
- How we can control N^2 LEDs using $2N$ GPIO pins using *multiplexing*.
- How we can set a single LED in a multiplexed grid.
- How we can create arbitrary patterns by quickly flashing LEDs on/off.
- How to configure GPIO.
- How to use pushbuttons.